

Digital Signal Processing with FAUST

Xianglei Wu



BACHELORARBEIT

eingereicht am

Fachhochschul-Bachelorstudiengang

Software Engineering

in Hagenberg

im Mai 2026

Betreuung:

Erik Pitzer, FH-Prof. DI (FH) Dr.

© Copyright 2026 Xianglei Wu

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe <https://creativecommons.org/licenses/by-nc-nd/4.0/>.

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Die vorliegende, gedruckte Arbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Hagenberg, am 25. Mai 2026

Xianglei Wu

Inhaltsverzeichnis

Erklärung	iv
Preface	vii
Abstract	viii
Kurzfassung	ix
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Audio-DSP	4
2.1.1 Physikalische Grundlage des Audiosignals	4
2.1.2 Signal-Rausch-Verhältnis (SNR)	5
2.1.3 Abtastung und Quantisierung	5
2.1.4 Digitale Audiosignale als Sample-Folgen	5
2.2 Frequenzanalyse mit DFT und FFT	5
2.2.1 Zeitbereich und Frequenzbereich	5
2.2.2 Diskrete Fourier-Transformation	5
2.2.3 Fast Fourier Transformation	5
2.2.4 Amplituden- und Leistungsspektrum	5
2.2.5 Frequenzauflösung	5
2.3 Kurzzeitanalyse und STFT	5
2.3.1 Nichtstationarität von Sprache	5
2.3.2 Frame-basierte Verarbeitung	5
2.3.3 Short-Time Fourier Transform	5
2.3.4 Magnitude und Phase	5
2.4 Fensterfunktionen	5
2.4.1 Blockbildung und Randdiskontinuitäten	5
2.4.2 Spectral Leakage	5
2.4.3 Hanning- und Hamming-Fenster	5
2.4.4 Bedeutung für spektrale Subtraktion	5
2.5 Overlap-Add und Rekonstruktion	5

2.5.1	Inverse FFT	5
2.5.2	Überlappende Frames	5
2.5.3	Rekonstruktion des Zeitsignals	5
2.6	Echtzeitanforderungen	5
2.6.1	Latenz	5
2.6.2	Puffergröße	5
2.6.3	Rechenzeit pro Frame	5
2.6.4	Overrun und Underrun	5
2.6.5	Echtzeitfähigkeit der spektralen Subtraktion	5
2.7	Zusammenfassung	5
3	Working with LaTeX	6
4	Figures, Tables, Source Code	7
5	Mathematical Elements	8
6	Using Literature	9
7	Printing the Manuscript	10
8	Closing Remarks	11
A	Technical Details	12
B	Supplementary Materials	13
B.1	PDF Files	13
B.2	Media Files	13
C	Questionnaire	14
D	LaTeX Source Code	15
	Quellenverzeichnis	16
	Literatur	16

Preface

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik besonderen Wert in Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik, die mit hohen Anforderungen an Laufzeiteffizienz, deterministische Latenz und numerische Präzision. Grundsätzlich sind die Implementierungen der DSP-Algorithmen in C oder C++, welches aufgrund deren direkten Zugriff auf Hardware Ressourcen und deren extrem effizienten Übersetzer (GCC, Clang) hochgradig optimierte Maschinencode erzeugt, für die Bachelorarbeit die erste Wahl. Der Knackpunkt trotz all den Vorteilen liegt an der doch noch sehr zeitintensive, fehleranfällige mathematische Übersetzung bis hin zu manuelle Implementierung von DSP-Algorithmen und das exportieren auf verschiedenen Zielplattformen, wie auf Desktop, verschiedensten Embedded Systemen, Web und Plug-in-Hosts; Daraus kristallisierten sich in den letzten zwei Jahrzehnten domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung, welches durch Abstraktion und Code-Generierung die Probleme bewältigen soll.

Die domänenspezifische Sprache Faust (Functional Audio Stream), entwickelt am GRAME in Lyon von, verspricht einen Lösungsansatz zu den erwähnten Herausforderungen. Faust ist eine funktionale, blockbasierte Programmiersprache, die speziell für Audio-DSP entworfen wurde. Programme werden als algebraische Komposition von Signalverarbeitungsoperatoren formuliert. Der Faust-Compiler übersetzt die mathematisch knappe Beschreibung eines Signalflussdiagramms in mehrere Zielsprache, darunter C++, Rust, JavaScript und WebAssembly. Aus einer einzigen Quellbeschreibung lassen sich somit Plug-ins für Digital Audio Workstations, Embedded-Anwendungen auf Mikrocontrollern und Webapplikationen erzeugen. [0]

Um für die industrielle Tauglichkeit muss der durch Faust generierte Code sowohl funktional korrekt, als auch laufzeiteffizient sein. Befürworter von Faust argumentieren, dass der generierte Code durch globaler Optimierung über den vollständigen Signalflussgraphen, automatischer Vektorisierung [0] und Parallelisierung [0] mit handgeschriebener C++ Code ebenbürtig oder dieses sogar übertreffen kann. Diese Behauptung vertreten auch die Entwickler in der offiziellen Faust-Dokumentation. [0] Ob handgeschriebener, manuell optimierter C++-Code grundsätzlich performanter ist

als Faust-generierter Code, ist in der wissenschaftlichen Literatur bislang nur punktuell untersucht. Eine systematische Literaturrecherche zeigt zwei peer-reviewte Studien mit direktem Performanz-Vergleich zwischen Faust-generiertem und handgeschriebenem C++-Code.

[0] berichten für den Reverb-Algorithmus *Freeverb* einen Geschwindigkeitsvorteil zugunsten von Faust von 28 Prozent und für den Multi-Tap-Delay *Tapiir* von 49 Prozent. [0] erweitern diesen Vergleich auf neun Instrumente des *Synthesis Toolkit* (STK) und ermitteln dort Geschwindigkeitsvorteile zwischen 18 und 80 Prozent. Beide Studien stammen aus dem erweiterten Faust-Umfeld und untersuchen ausschließlich Algorithmen im Zeitbereich (Reverbs, Delays, physikalische Modelle).

Für FFT-intensive DSP-Algorithmen liegt keine vergleichende Untersuchung vor. Diese Lücke ist nicht zufällig: Die Faust-Dokumentation räumt selbst ein, dass die Sprache durch ihr sample-orientiertes Berechnungsmodell strukturelle Schwierigkeiten mit Multi-Rate-Algorithmen wie FFT und Faltung aufweist [0]. Da gerade FFT-basierte Verarbeitung den Kern vieler praxisrelevanter DSP-Anwendungen –etwa Sprachverbesserung, Audio-Codecs und akustische Analyse –bildet, ergibt sich hier eine wissenschaftlich wie praktisch relevante Forschungslücke.

Die vorliegende Arbeit setzt an dieser doppelten Lücke an: Sie untersucht systematisch und mit differenzierter Methodik, wie sich Faust-generierter C++-Code gegenüber einer manuell implementierten C++-Lösung verhält, wenn beide Implementierungen einen FFT-basierten Algorithmus realisieren. Als Anwendungsfall wird ein klassisches, in der Literatur umfassend dokumentiertes Verfahren der Einkanal-Rauschunterdrückung herangezogen – *Spectral Subtraction* nach Boll. [0] Die Wahl eines etablierten, wohlverstandenen Algorithmus erlaubt es, den Fokus konsequent auf den Sprach- und Implementierungsvergleich zu legen.

1.2 Forschungsfrage und Zielsetzung

Ziel dieser Arbeit ist es an dieser doppelten Lücke anzusetzen. Sie soll die Untersuchung der Eignung von Faust für die Implementierung von FFT-intensiven Echtzeit-DSP-Anwendungen im direkten Vergleich zu einer äquivalenten manuellen nicht optimierten C++-Implementierung zeigen. Dabei soll die Handhabung und Erlernbarkeit der Sprache Faust bei der Implementierung berücksichtigt werden.

Zur Beantwortung der Hauptaufgabe werden folgende Teilfragen untersucht:

- Wie groß ist der Performanz-Unterschied(CPU-Zeit pro Audio-Block) zwischen Faust-generiertem und handgeschriebenem Code für elementare DSP-Bausteine(Gain, Biquad-Filter, FIR, FFT)?
- Wie skaliert dieser Unterschied bei der Verknüpfung zu einer vollständigen Pipeline(Framing, Fensterung, FFT, Spectral Subtraction)?
- Bestehen Objektive Qualitätsunterschiede(PESQ, STOI, SNR-Verbesserung) zwischen den beiden Implementierungen bei identischem Algorithmus?
- Developer Experience: Wie bewährt sich Faust hinsichtlich Entwicklerfreundlichkeit, Codelänge, Wartbarkeit, Komplexität im Vergleich zum Aufbau komplexer Datenstrukturen in C++?

1.3 Aufbau der Arbeit

Die vorliegende Bachelorarbeit ist methodisch in acht Kapitel strukturiert. Der Prozess umfasst, theoretische Grundlagen der DSP, Aufbau des Prototyps, die Durchführung systematischer Vergleiche sowie die abschließende Evaluierung.

Nach dieser Einleitung wird im Kapitel 2 (die Grundlagen) für das notwendige Verständnis der Digitale Signalverarbeitung erläutert, genaueres über Audio-DSP, Fast Fourier Transformation, Fensterfunktionen und sowie generell Echtzeitanforderungen.

Darauf aufbauend stellt Kapitel 3 (Faust) die domänenspezifische Sprache im Detail vor. Einblicke in die funktionale Syntax und Semantik sowie die zugrundeliegende Compiler-Architektur. In Kapitel 4 (Spectral Subtraction) wird die ausgewählte Algorithmus, ein Rauschunterdrückungsverfahren nach Boll theoretisch hergeleitet und auf deren Parametern genauer eingegangen.

In Kapitel 5 (Implementierung) wird die praktische Umsetzung beider Varianten beschrieben. Faust-Code, C++-Code, gemeinsame Schnittstellendesign und sowie Build-System für eine faire Vergleichsbasis werden dokumentiert.

Die Beantwortung der Forschungsfragen erfolgt in Kapitel 6 (Evaluation). In diesem Kapitel werden die Versuchsaufbau detailliert dargelegt und die Ergebnisse der Mikro- und Makro-Benchmarks (Laufzeit, Speicher, Binärgröße) sowie der Qualitätsmessungen (PESQ, STOI) auf beiden Testplattformen ausgewertet.

Danach wird in Kapitel 7 (Diskussion) die erhobene Daten interpretiert. Es werden Limitationen der Untersuchungen, die Aussagekräftigkeit der Durchführungen und die qualitativen Aspekte der Entwicklererfahrung beider Sprachen gegenübergestellt.

Zum Abschluss mit Kapitel 8 (Fazit und Ausblick) wird die Arbeit zusammengefasst und ein Ausblick auf mögliche Erweiterungen, oder Lücken hingewiesen.

Kapitel 2

Grundlagen

2.1 Grundlagen der Audio-DSP

2.1.1 Physikalische Grundlage des Audiosignals

In der Signalverarbeitung wird die Erfassung, Speicherung, Anzeige und Erzeugung von Signalen von Audiosignalen gesprochen. Ebenso als Extraktion und Neu-Codierung von Informationen. In der Regel kommt die Quelle für die Signalverarbeitung durch mechanische Schwingungen in der Luft ein Schall mit einer Schallgeschwindigkeit $c \approx 343 \text{ m/s}$ im Raum ausbreitet. Um dies zu erfassen ist die einfachste Darstellung eines Signals die Sinuswelle. [0]

$$x(t) = A \cdot \sin(\omega t + \phi) \quad (2.1)$$

Die wichtigsten Größen im Zeitbereich sind:

- **Amplitude** A : die Größe der Schwingung, also die maximale Druckabweichung vom Ruhezustand.
- **Frequenz** f : die Anzahl der Schwingungswiederholungen pro Sekunde, gemessen in Hertz (Hz).
- **Periode** T : die Dauer einer vollständigen Schwingung, mit

$$T = \frac{1}{f} \quad (2.2)$$

- **Phase** ϕ : die Anfangslage der Schwingung im Zyklus, angegeben in Radiant.

2.1.2 Signal-Rausch-Verhältnis (SNR)

2.1.3 Abtastung und Quantisierung

2.1.4 Digitale Audiosignale als Sample-Folgen

2.2 Frequenzanalyse mit DFT und FFT

2.2.1 Zeitbereich und Frequenzbereich

2.2.2 Diskrete Fourier-Transformation

2.2.3 Fast Fourier Transformation

2.2.4 Amplituden- und Leistungsspektrum

2.2.5 Frequenzauflösung

2.3 Kurzzeitanalyse und STFT

2.3.1 Nichtstationarität von Sprache

2.3.2 Frame-basierte Verarbeitung

2.3.3 Short-Time Fourier Transform

2.3.4 Magnitude und Phase

2.4 Fensterfunktionen

2.4.1 Blockbildung und Randdiskontinuitäten

2.4.2 Spectral Leakage

2.4.3 Hanning- und Hamming-Fenster

2.4.4 Bedeutung für spektrale Subtraktion

2.5 Overlap-Add und Rekonstruktion

2.5.1 Inverse FFT

2.5.2 Überlappende Frames

2.5.3 Rekonstruktion des Zeitsignals

2.6 Echtzeitanforderungen

2.6.1 Latenz

2.6.2 Puffergröße

2.6.3 Rechenzeit pro Frame

2.6.4 Overrun und Underrun

2.6.5 Echtzeitfähigkeit der spektralen Subtraktion

2.7 Zusammenfassung

Kapitel 3

Working with LaTeX

Kapitel 4

Figures, Tables, Source Code

Kapitel 5

Mathematical Elements, Equations and Algorithms

Kapitel 6

Using Literature and other Resources

Kapitel 7

Printing the Manuscript

Kapitel 8

Closing Remarks

Anhang A

Technical Details

Anhang B

Supplementary Materials

List of supplementary data submitted to the degree-granting institution for archival storage (in ZIP format).

B.1 PDF Files

Pfad: /

thesis.pdf Master/Bachelor thesis (complete document)

B.2 Media Files

Pfad: /media

*.ai, *.pdf Adobe Illustrator files
*.jpg, *.png raster images
*.mp3 audio files
*.mp4 video files

Anhang C

Questionnaire

Anhang D

LaTeX Source Code

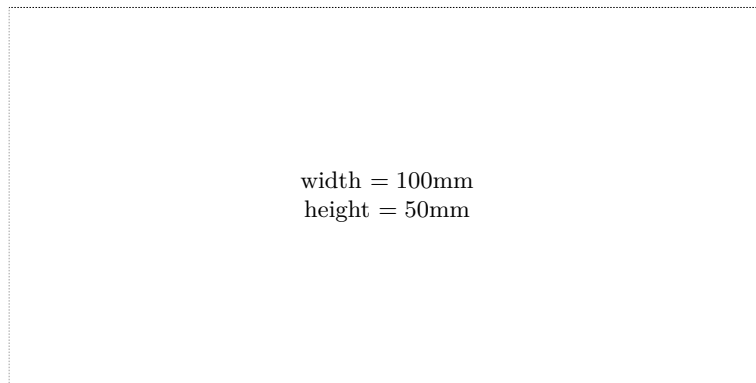
Quellenverzeichnis

Literatur

- [0] Steven Boll. „Suppression of acoustic noise in speech using spectral subtraction“. *IEEE Transactions on acoustics, speech, and signal processing* 27.2 (1979), S. 113–120.
- [0] Stéphane Letz, Yann Orlarey und Dominique Fober. „Work Stealing Scheduler for Automatic Parallelization in Faust“. In: *Proceedings of Linux Audio Conference*. Hrsg. von LAC. Utrecht, Netherlands, 2010. URL: <https://hal.science/hal-02158924> (siehe S. 1).
- [0] Romain Michon und Julius O. Smith. „FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language“. In: *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)*. Paris, France, Sep. 2011 (siehe S. 2).
- [0] Yann Orlarey, Dominique Fober und Stéphane Letz. „FAUST : an Efficient Functional Approach to DSP Programming“. In: *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC*. Hrsg. von Editions DELATOUR FRANCE. 2009, S. 65–96. URL: <https://hal.science/hal-02159014> (siehe S. 1, 2).
- [0] Yann Orlarey, Dominique Fober und Stéphane Letz. „FAUST : an Efficient Functional Approach to DSP Programming“. In: *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC*. Hrsg. von Editions DELATOUR FRANCE. 2009, S. 65–96. URL: <https://hal.science/hal-02159014> (siehe S. 2).
- [0] T. Rossing. *Springer Handbook of Acoustics*. Springer Handbook of Acoustics. Springer New York, 2007. URL: https://books.google.at/books?id=4ktVwGe_dSMC.
- [0] Nicolas Scaringella u. a. „Automatic vectorization in Faust“. In: *Actes des Journées d’Informatique Musicale JIM2003*. Hrsg. von JIM. Montbeliard, France, 2003. URL: <https://hal.science/hal-02158949> (siehe S. 1).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —